

# Building a system, learning to understand it

*Personal contribution and reflection*

**Xuanye(Sawyer) Xu**

Student ID: H00490299 / 24020036071

12 September 2026

My ambition was to build more than a working demonstration. I wanted a system that an ordinary user could start with a click. Developing the RoboMaster EP task showed me the distance between making a robot move, understanding its behaviour, and making it easy for someone else to use.

## 01 My role and contribution

My responsibilities covered the simulation and hardware programs, physical debugging, and guidance on the grasping scene. I participated throughout development: defining the fixed-base task, discussing model choices, preparing the remote environment, observing tests, and refining the pickup and placement conditions.

I helped translate the broad task into concrete decisions: an upright bottle on the ground, 150 ml of water, fixed points A and B, a safe lifting height, and a home-pick-place-return sequence. During hardware testing, I positioned the bottle, operated the terminal, observed the arm, and compared its behaviour with the logs. This feedback informed changes to the target positions and motion checks.

AI assisted substantially with code generation and revision. My contribution included specifying requirements, providing physical observations, reviewing proposed actions, and deciding whether the result matched the task. I also participated in organising configurations, test records, videos, and the GitHub submission.

## 02 Improving the way I worked

Using SSH and VNC to access the Jetson from my Mac was an important practical improvement. SSH supported terminal operations, while VNC provided access to the graphical desktop. This reduced the repeated setup of a separate monitor and mouse and made development less dependent on a fixed workstation. I learned that efficiency also comes from improving the working environment, not only from writing code faster.

---

### THE STARTING POINT

A working prototype was a milestone. A usable product was the larger ambition.

---

### DIVISION OF WORK

My account distinguishes responsibility for development and verification from independently writing every line of code.

---

### A PRACTICAL LESSON

Removing repeated setup made it easier to move between development and observation.

### 03 When code grew faster than understanding

AI coding tools were powerful, but they also exposed a gap in my understanding. Generated programs and configuration files could accumulate much faster than I could explain them. Faced with unfamiliar ROS 2 concepts, SDK calls, and communication settings, I sometimes felt that I was reading a foreign language. That confusion could turn into self-doubt, even when development appeared to be moving quickly.

The gripper-release problem made this lesson concrete. From observation, it looked as if the robot simply would not open its gripper. The logs showed that an earlier movement had failed, so execution had never reached the release step. Another return-home attempt failed during connection establishment. Similar visible outcomes therefore had different causes.

I began to see why understanding program structure matters. I needed to distinguish a command being sent, the SDK reporting completion, and feedback confirming actual movement. I also needed to understand which file defined a position and which program executed it. AI could help explain and change these parts, but reading, remembering, testing, and making connections remained my responsibility.

*The code could grow faster than my understanding. Progress required me to close that gap.*

### 04 From prototype to product

The final system remained closer to an experimental prototype than to the click-to-use product I imagined. Time, available energy, the pace of learning, and team coordination all limited what could be completed. However, I do not want to explain every limitation through external constraints.

I also recognise my own tendency to postpone difficult learning and documentation. Sometimes I preferred the immediate satisfaction of seeing a function run to the slower work of understanding it. That avoidance allowed technical debt and gaps in knowledge to accumulate. A usable product requires attention to setup, clear controls, understandable errors, and recovery procedures, not just a successful demonstration.

In a future project, I would first understand a minimal working system, document each completed function, and keep a short record of failures and decisions. I would design the ordinary user's workflow earlier, rather than leave usability until the end.

*My main gain was a clearer sense of ownership: using powerful tools effectively means remaining accountable for what the system does, what I understand, and what still needs improvement.*

---

#### THE TURNING POINT

"The gripper did not open" described the symptom. Finding the failed stage explained why.

#### WHAT I MUST RETAIN

The roles of ROS 2, the SDK, communication, configuration, and feedback cannot remain a collection of unexplained commands.

---

#### NEXT TIME

Understand the smallest working system.

Record decisions as they happen.

Design the user's experience from the beginning.